TECHNICAL DESIGN DOCUMENT

NESTORA

Smart Township Management System — smart township management for a gated Pune community, with MySQL persistence and Groq-assisted complaint triage.

GitHub repository	https://github.com/Sarancode6369/NESTORA
Live application	https://nestora-gray.vercel.app/

Version	2.0 (database migration & AI integration)
Original date	4 November 2026
Revised	5 November 2026 (testing results, local MySQL verification, and AI and production status updated; structure and screenshots retained)
Scope	Architecture, schema, flows, AI workflow, safety handling, security, testing, mobile layout
Status	**Implemented.** Local MySQL 8.0.46 integration, database initialization, schema verification, and application health check successfully tested. Automated test suite: 98 passed, 0 failed, 1 skipped. Live Groq API integration was not verified.
Verification scope	**Local MySQL:** verified (§12.3). **Aiven/Vercel production database:** not verified; the local MySQL results do not establish that the production database connection works (§12.5). **Live Groq API and production AI integration:** not verified (§12.4, §12.5).

Contents

1. Project Overview
2. System Architecture and Technology Stack
3. Database Design and Data Model
4. Authentication and Role-Based Access
5. Resident Dashboard and User Interface
6. Complaint Management and Help Desk
7. Visitor Management and Gate Security
8. Maintenance Payments and Financial Management
9. Amenity Booking and Other Modules
10. AI Functionality and Automation
11. Security, Privacy and UX Decisions
12. Testing and Quality Assurance
13. Assumptions, Limitations and Future Improvements
14. Mobile Responsive Design

1. Project Overview

NESTORA replaces Excel maintenance bills, manual payment matching, WhatsApp complaint groups and paper visitor registers for a township of 2,400 apartments in 28 towers served by 180 staff. Monthly maintenance is flat area × ₹6.50.

GitHub **https://github.com/Sarancode6369/NESTORA**

Live application **https://nestora-gray.vercel.app/**

This revision (a) replaces the JSON-file store with MySQL, (b) replaces the keyword-only complaint classifier with an LLM classifier (Groq) that is validated, bounded and always backed by a deterministic fallback, and (c) keeps the existing UI, branding and workflows intact.

Objectives

Goal	How it is met
Relational persistence	26 normalised InnoDB tables, foreign keys, indexes, CHECK and UNIQUE constraints (§3)
Money safety	One transaction per state-changing request; database-level uniqueness for payment references, receipts, bank transactions and UTRs
Trustworthy AI	Schema-validated output, one safe retry, keyword fallback, human review queue, audit of corrections (§10)
Emergency safety	Rule engine independent of the model; the model can raise but never lower urgency (§10)

Features and scope

The modules documented in this report are: OTP sign-in with role-based access (§4); the resident home and dashboard (§5); complaints and the help desk with AI-assisted triage (§6, §10); visitor management at the gate (§7); maintenance bills, ledger, receipts, bank reconciliation and committee analytics (§8); and amenity booking, notices and notifications (§9). The application is a progressive web app in English, Hindi and Marathi, with a mobile layout shown in §14.

2. System Architecture and Technology Stack

Browser · PWA · EN/HI/MR → HTTPS + JWT → HTTP API (app.js) → Services → Repositories (SQL) → MySQL 8

Services → AI layer → Groq API → **on any failure** → Keyword classifier

Decision	Choice	Reason
Runtime	Node.js 22, ES modules, no framework	Retains the existing working codebase and avoids a rewrite. Built-in `fetch` and `node:test` keep dependencies to one (`mysql2`).
Database	MySQL 8, InnoDB, utf8mb4	Required by the brief; transactional, foreign keys, CHECK constraints (8.0.16+).
Data access	Repository layer with parameterised SQL; no ORM	All SQL in `server/repos` ; services never build SQL; injection-safe by construction.
Engine abstraction	`server/db` adapter: MySQL (production), SQLite via `node:sqlite` (tests, `dev:sqlite`)	The same `schema.sql` runs on both so tests exercise real constraints and transactions where MySQL is unavailable. SQLite is never selected unless explicitly requested.
Identifiers	Readable ids (`CMP-0001` , `BILL-0001`) from a counters table	Preserves existing ids and receipts; the counter row is locked for the transaction so ids are gap-free and race-safe.
Time / money	UTC `DATETIME(3)` , `DATE` , `DECIMAL(12,2)`	No floating-point money or timezone drift (`dateStrings` , `decimalNumbers`).
Auth	HS256 JWT, role claims, demo OTP	Mock OTP per the brief; roles enforced per route.
AI provider	Groq Chat Completions, JSON mode, temperature 0	Low latency, structured output; model is configurable (`GROQ_MODEL`) because hosted model catalogues change.

> **Stack note.** An earlier requirements document named Spring Boot and React. The delivered, working system is Node and vanilla-JS; this upgrade keeps it rather than rewriting working features. The database, API contracts and layering map one-to-one to a Spring Data JPA port should that be required.

3. Database Design and Data Model

Created by `database/schema.sql` (idempotent, recorded in `schema_migrations`). Sample data is loaded by `scripts/db.js seed`; legacy JSON is imported by `import-json` (taxonomy mapped Housekeeping→Cleaning, Critical→Emergency).

Group	Tables	Notes
Reference	`roles`, `app_settings`, `counters`, `schema_migrations`	Rate, due days and late-fee policy live in `app_settings` — configurable, never hard-coded.
People & property	`flats`, `flat_upi_ids`, `users`, `staff`	UPI id → flat is UNIQUE; phone UNIQUE; user→role and user→flat FKs.
Billing	`maintenance_bills`, `flat_ledger`, `payment_sessions`	One bill per (flat, period). Ledger rows are bills, late fees and payments.
Reconciliation	`bank_statements`, `bank_transactions`	Status is one of matched, partial, unmatched, needs_review, duplicate; reasons and confidence stored.
Complaints	`complaints`, `complaint_photos`, `complaint_history`, `complaint_corrections`, `complaint_ratings`	Classification result, method, model, confidence, explanation, review flag and safety rules stored on the complaint; self-FK `linked_to` for duplicates.
Community	`visitors`, `guest_codes`, `notices`, `amenities`, `bookings`, `enquiries`	`visitors.client_id` UNIQUE makes offline replay idempotent; `bookings.slot_key` UNIQUE prevents double booking.
Governance	`notifications`, `audit_logs`	Append-only trail of financial, reconciliation, correction and retention events.

Constraints that protect money

Constraint	Prevents
`flat_ledger.payment_ref` UNIQUE (normalised)	Posting the same payment reference or UTR twice
`flat_ledger.receipt_no` UNIQUE · `flat_ledger.bank_txn_id` UNIQUE	Duplicate receipts; one bank transaction credited to two ledgers
`bank_transactions.utr_key` UNIQUE (set only on the canonical row)	Two live rows for one UTR. Re-imports are stored with status duplicate and a NULL key.
`maintenance_bills (flat_no, bill_period)` UNIQUE	Double billing a month
CHECK: amount ≥ 0, enums for status/category/priority/method, confidence 0–1	Invalid values reaching storage regardless of application bugs

Indexes cover the hot paths: ledger by (flat, date) and (kind, date); complaints by status, (category, priority), (assignee, status), created time and review flag; bank transactions by status, fingerprint and flat; audit by time and action.

4. Authentication and Role-Based Access

4.1 Authentication workflow

`Enter linked mobile number` → `Send OTP (demo: shown on screen, no SMS)` →
`Enter 6-digit OTP` → `Verify` → `HS256 JWT (8 h)` → `Role-specific home`

A user signs in with the mobile number linked to their flat or staff record (Figure 1). The server issues a one-time password; in the demonstration build the OTP is displayed on the verification screen and no SMS is sent (Figure 2). A successful verification returns a signed JWT with role claims, valid for 8 hours. Both screens carry the demonstration-mode notice.

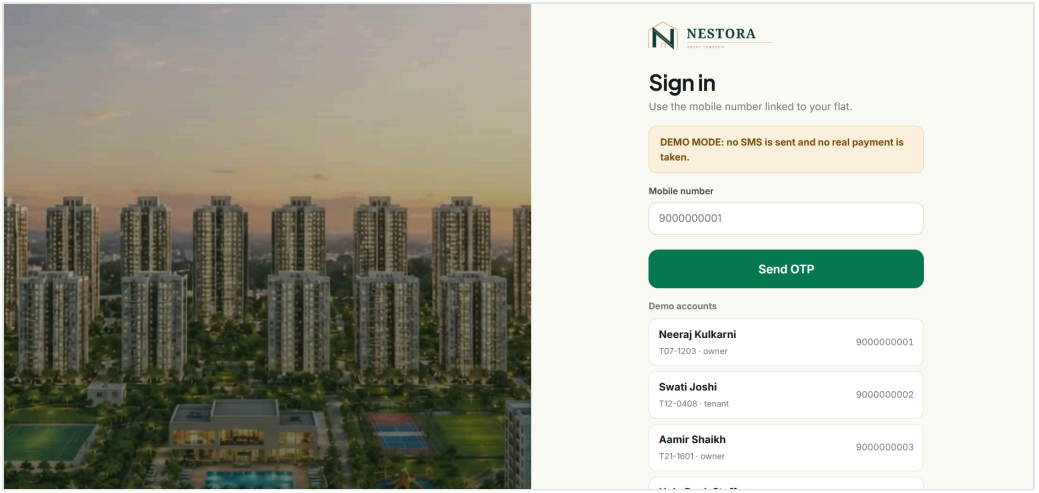


Figure 1: NESTORA Sign-in Screen – Mobile Number Entry with Demo Mode Notice.

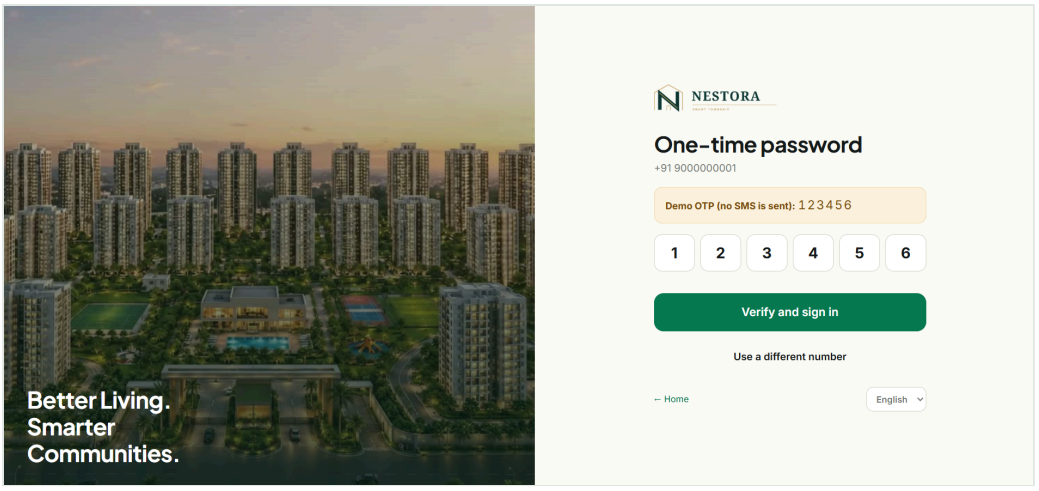


Figure 2: NESTORA OTP Verification – Demo OTP (No SMS Sent) and Six-Digit Entry.

4.2 Roles and access permissions

Role	Access (as visible in the delivered interface and the security controls)
Resident	Home, Pay, Complaints, Visitors, Book. Confined to their own flat; cross-flat access is tested and rejected.
Help desk	Help desk, Payments, Dashboard, Admin navigation.
Committee	Committee dashboard with collections, ageing and monthly CSV report (§8).
Guard	Gate screen for visitor registration and entry/exit (§7).

Every route declares its allowed roles. A route × role matrix test checks each route (403 for the wrong role, 401 without a valid token), and JWT tampering and expiry are tested (§12).

5. Resident Dashboard and User Interface

The public landing page introduces the township and leads to sign-in (Figure 3). It shows the navigation (Home, 360°, About, Services, Amenities, Notices, Contact), the language selector, a Sign In button and the headline statistics: 2,400 residences, 28 towers and 180 staff.

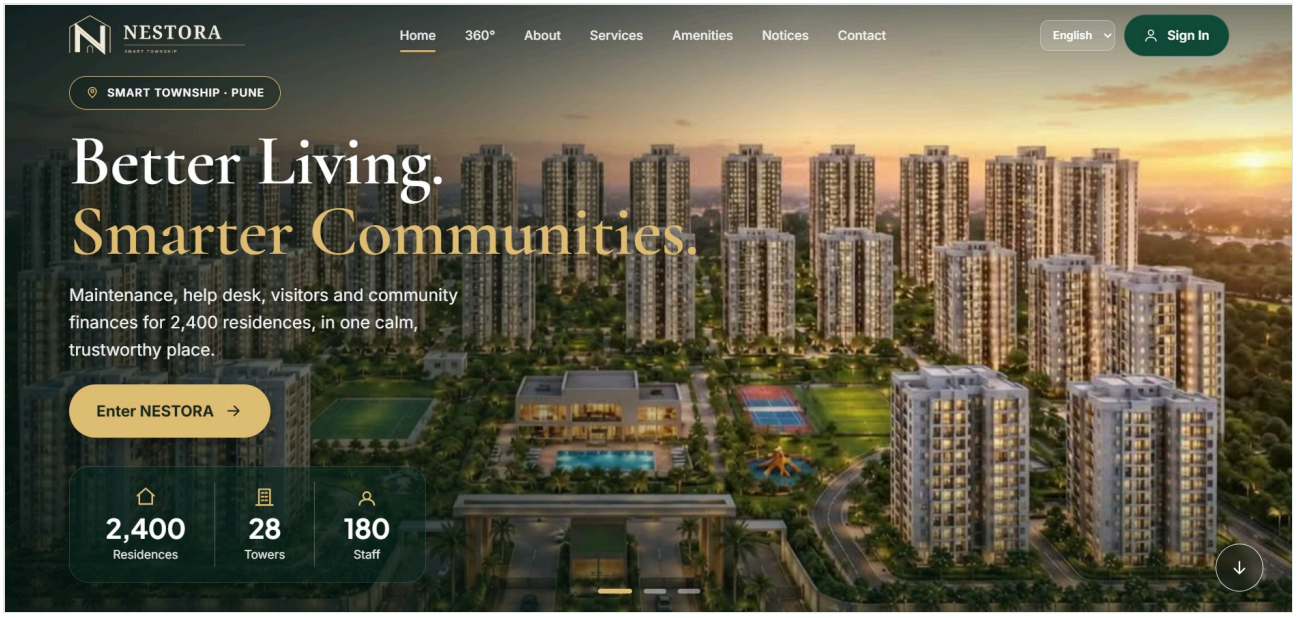


Figure 3: NESTORA Home Page – Smart Township Landing Screen with Navigation and Sign-In.

Resident navigation and features

After sign-in, residents use five primary destinations: **Home**, **Pay**, **Complaints**, **Visitors** and **Book**. The resident home shows maintenance due, open complaints and visitors today, followed by quick actions (see Figure 9 in §14). A desktop capture of the resident dashboard was not supplied for this revision, so the dashboard layout is documented with the mobile capture only.

- **Pay** — bills, ledger and receipts (§8).
- **Complaints** — raise and track complaints (§6).
- **Visitors** — approve or deny visitors waiting at the gate (§7).
- **Book** — amenity slot booking (§9).

Design principle: the most important information is within two taps, and every automated decision is visible and reversible by a person (§11).

6. Complaint Management and Help Desk

6.1 Submission

A resident raises a complaint with a title, a description, an optional category and up to three photos (Figure 4). The category is optional because the classifier (§10) determines category and priority from the text.

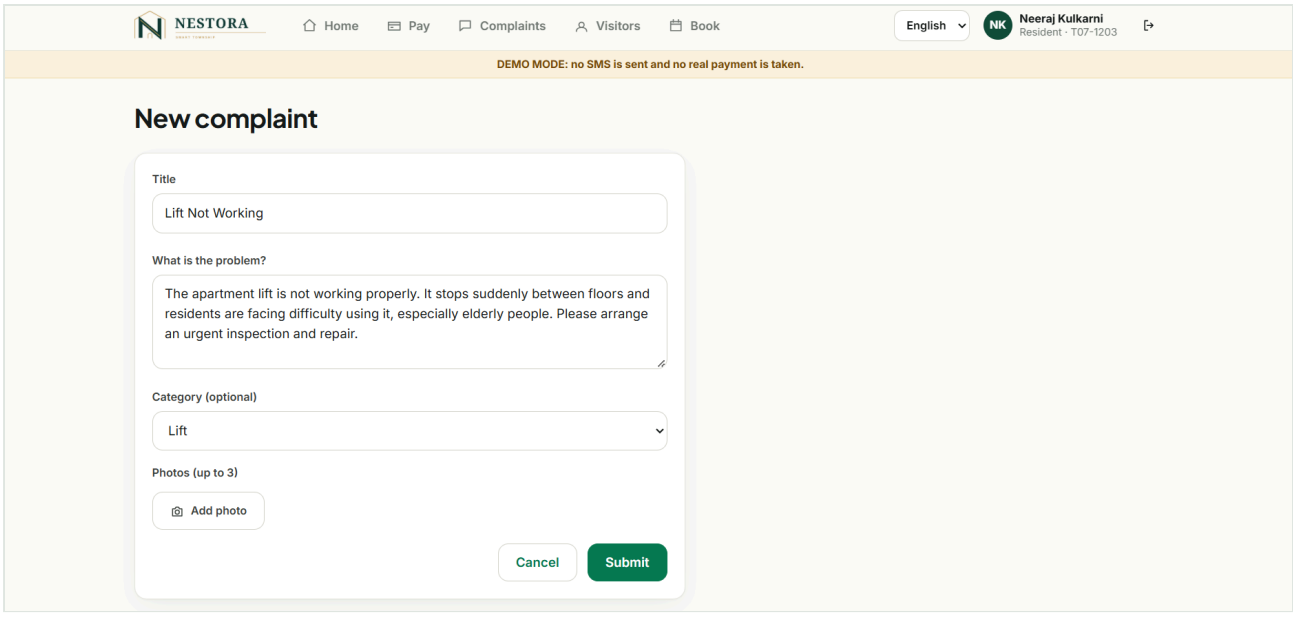


Figure 4: NESTORA New Complaint Form – Title, Description, Optional Category and Photo Upload.

6.2 Complaint lifecycle

Resident submits → Classify (§10) → Auto-suggest staff → Emergency? duty manager → Duplicate link → Help desk: correct / assign / status → Resolved by a person → Rate or reopen

6.3 Help desk dashboard

Help-desk staff see summary tiles (Open, Needs review, Response overdue, Emergency), filters by status, category and urgency, a “Needs review only” toggle, and a table of complaints with flat, category, urgency, status, assigned staff and response-time indicator (Figure 5). Statuses visible in the capture include Reopened, Assigned and Resolved. The records shown are demonstration data.

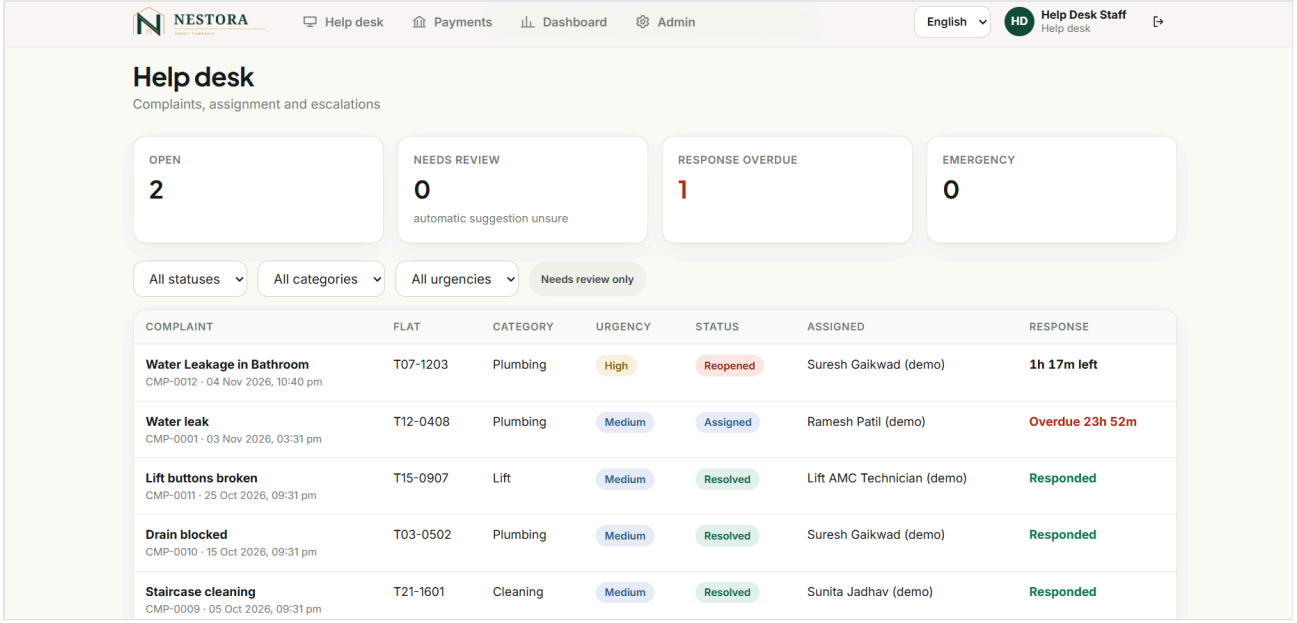


Figure 5: NESTORA Help Desk Dashboard – Complaint Queue, Filters, Assignment and Response Tracking.

In the complaint drawer the help desk sees the classification method badge (AI / keyword fallback / corrected by staff), confidence and explanation, a manual-review banner with a one-tap Confirm, a priority control and the correction audit trail. Residents never see AI internals. Complaints are never auto-closed, and duplicates are linked to the original and stay visible and open.

7. Visitor Management and Gate Security

The gate screen is a dedicated dark interface for guards (Figure 6). It offers a **New visitor** action for registration and a **Guest code** action, a **Waiting for resident** list and an **Inside now** list, and an Online indicator.

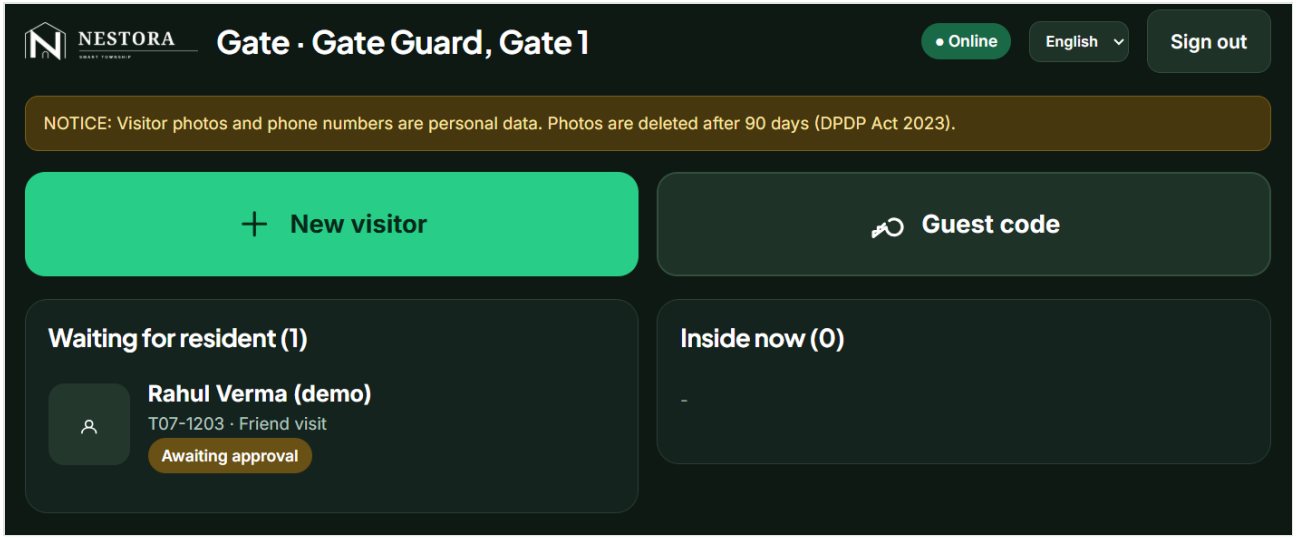


Figure 6: NESTORA Gate Guard Dashboard – Visitor Registration, Resident Approval and Entry Monitoring.

Guard registers visitor → Waiting for resident → Resident approves or denies → Guard records entry → Inside now → Guard records exit

- **Visitor registration.** The guard registers the visitor against a flat and purpose; in the capture, a demo visitor for flat T07-1203 (“Friend visit”) is listed as *Awaiting approval*.
- **Resident approval.** The resident approves or denies; the guard then records entry and exit.
- **Waiting and inside lists.** “Waiting for resident (1)” and “Inside now (0)” show current counts.
- **Offline behaviour.** If the guard device is offline, registrations are queued locally and replayed idempotently by `clientId` (`visitors.client_id` is UNIQUE).
- **Guest code.** A Guest code action and a `guest_codes` table exist; the original document does not describe the code rules in more detail, so none are claimed here.

Privacy notice and retention

The gate screen displays a notice that visitor photos and phone numbers are personal data and that photos are deleted after 90 days (DPDP Act 2023). Deletion is performed by an hourly job and is audited. The audit trail records staff actions. No visitor-security feature beyond those described above is claimed.

8. Maintenance Payments and Financial Management

8.1 Resident payment interface

The Pay page has three tabs — **Bills**, **Ledger** and **Receipts** — and a **Pay now** button showing the outstanding amount (Figure 7). Each bill row shows the month, the calculation basis, the amount, the amount paid, the balance and a status badge.

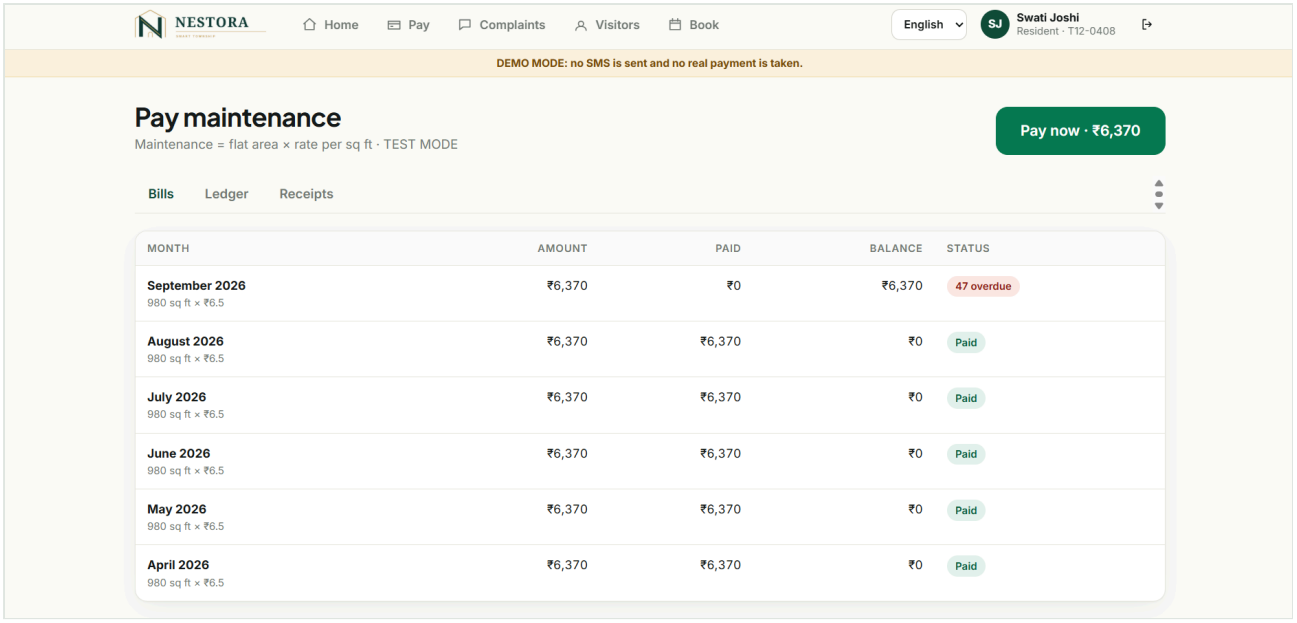


Figure 7: NESTORA Resident Payment Interface – Maintenance Bills, Payment Status and Ledger.

8.2 Maintenance calculation

Maintenance = flat area × rate per sq ft. For the flat shown (980 sq ft at ₹6.50), each monthly bill is 980 × 6.50 = ₹6,370. In the capture, the September 2026 bill has ₹0 paid and a balance of ₹6,370 with an overdue badge, while August to April 2026 are fully paid with a ₹0 balance. The rate, due days and late-fee policy are configuration values in `app_settings`.

8.3 Payment workflow and test-mode limits

`Sign in (OTP)` → `Dashboard: dues` → `Pay (test mode)` → `Checkout session` → `Complete (compare-and-set)` → `Ledger + receipt in one transaction`

Partial payments are allowed; payments settle charges oldest-first. A completed session cannot be completed again.

> **Test mode.** Payments in this build are test-mode only. The interface states “TEST MODE” and “no real payment is taken”; no real money or card data is handled. A PCI-compliant payment gateway is future work (§13).

8.4 Bank statement reconciliation

Upload CSV → Parse & validate → Duplicate check (fingerprint + UTR) → Match: reference, UPI, amount → Matched / Partial / Needs review / Unmatched → Human approves → Ledger posting

> Exact amount alone never confirms a match. UPI + amount without a payment reference is *needs review*. Upload never posts. Only exact reference matches may be auto-posted, and only by an explicit staff action; every approval, rejection and auto-post is audited.

8.5 Committee dashboard and financial analytics

Collections, ageing, resolution time and reopen % are computed from ledger and complaint rows, with a monthly CSV export. The committee dashboard (Figure 8) shows total billed, collected, outstanding dues and collection rate, a monthly Billed-versus-Collected chart (June to November) and a dues-ageing breakdown. The page states that the figures are computed from the demonstration dataset of five flats and that records marked demo are fictional; the values in the capture (billed ₹2,26,980, collected ₹2,01,625 = 88.83%, outstanding ₹25,355 across 4 of 5 flats) are therefore illustrative, not township-wide actuals.

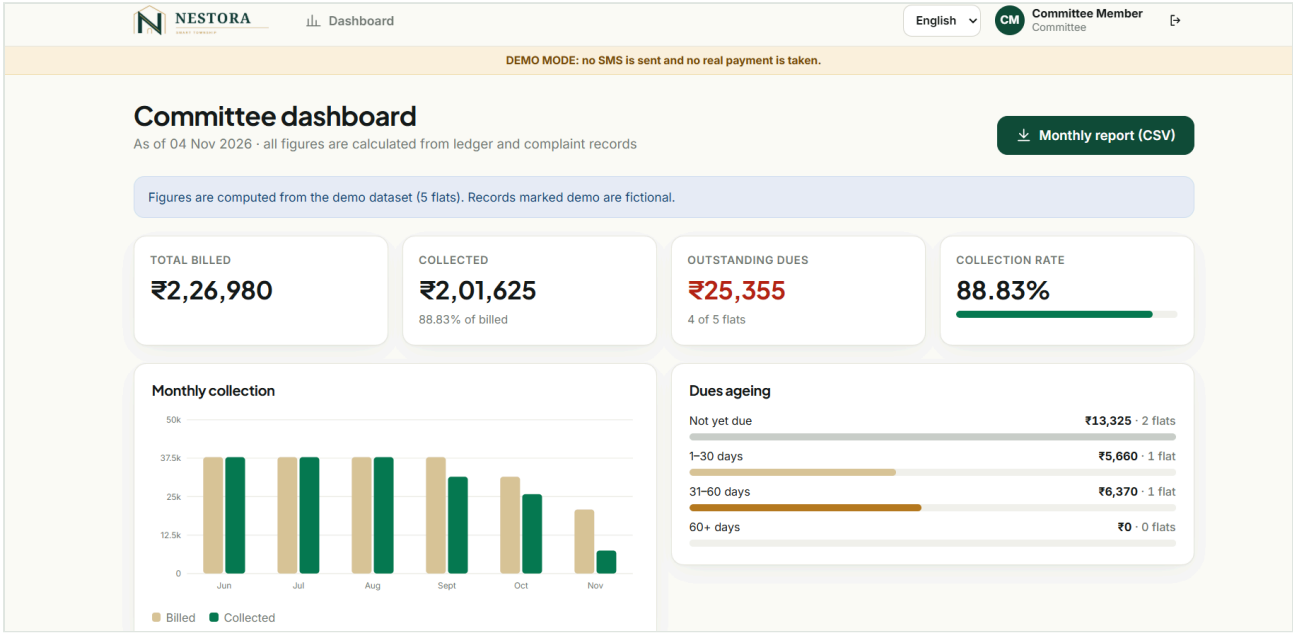


Figure 8: NESTORA Committee Dashboard – Billed, Collected and Outstanding Totals, Monthly Collection and Dues Ageing.

9. Amenity Booking and Other Modules

- **Amenity booking.** Slot availability is shown; one live booking per slot is enforced by a database constraint (`bookings.slot_key` UNIQUE); a 24-hour cancellation rule applies. Residents reach it from the Book tab.

- **Notifications.** Stored in the `notifications` table; for example, a notification is stored when a complaint is escalated as an emergency (§10).
- **Notices and enquiries.** Tables `notices` and `enquiries` support the public landing-page sections (Notices, Contact).
- **Audit.** `audit_logs` records financial, reconciliation, correction and retention events.

10. AI Functionality and Automation

Groq-assisted classification is an optional feature of NESTORA. It is implemented and configurable, but the live Groq API integration has not been verified (§12.4). Without a valid API key the application uses the keyword-based fallback classifier.

`1 Safety rules` → `2 Redact & build prompt` → `3 Groq (JSON, temp 0, timeout)` → `4 Validate` → `5 Merge with rules` → `6 Store + method`

Step	Detail
Input	Title and description only. Phone numbers, e-mail addresses, ID numbers, resident and flat identifiers are removed or never sent. The staff roster is sent as id, team and current load.
Output (required JSON)	`category` (Plumbing, Electrical, Lift, Security, Cleaning, Civil, Other), `priority` (Low, Medium, High, Emergency), `summary`, `suggested_department`, `suggested_staff`, `confidence` (0–1), `explanation`
Validation	Strict: enum membership, numeric range, bounded text, staff must be on duty in the chosen department. Anything else is rejected — the model's text is never stored unchecked.
Retry	An invalid answer is retried once with the validation errors fed back; a model that rejects JSON mode is retried once without it. A second failure → fallback.
Stored	`ai_category`, `ai_priority`, `ai_confidence`, `ai_explanation`, `classification_method` (`groq_ai` | `keyword_fallback` | `manual`), `classification_model`, `suggested_department`
Review	Confidence < `AI_MIN_CONFIDENCE` (0.6), AI/keyword disagreement, or a resident category the AI contradicts → `needs_review` with reasons; shown in the help-desk review queue.
Correction	Help desk changes category or priority in one action → method becomes `manual`, a row is added to `complaint_corrections` and `audit_logs`, history is appended.

10.1 Failure matrix (validation and fallback)

Condition	Behaviour	Complaint
No `GROQ_API_KEY`	Mock/fallback mode: keyword classifier, zero network calls	Created, `keyword_fallback`
Network error, timeout, 401, 429, 5xx, empty body	Keyword fallback; error recorded without key or complaint text	Created, `keyword_fallback`
Malformed or invalid JSON / unknown category	One retry, then fallback; bad data never stored	Created, `keyword_fallback`
Local call budget exceeded (`AI_MAX_CALLS_PER_MIN`)	Fallback without calling the API	Created
Low confidence	Kept, flagged for manual review	Review queue

10.2 Emergency rule (independent of the AI)

Regular-expression rules (English, Hindi, Marathi) detect fire, gas leak, lift stuck/trapped and electric shock. They run first and their result is applied after the model:

- Priority is forced to Emergency, status becomes `escalated`, the complaint is assigned to the duty manager and a notification is stored.
- The model's verdict is merged one-way: it may raise a complaint to Emergency (escalated too, but correctable) and can never lower a rule-triggered emergency — even if it returns Low, errors, or returns junk. Instructions embedded in complaint text (prompt injection) cannot disable this.
- Help desk cannot lower a rule-triggered emergency; only a person resolves it. Complaints are never auto-closed.
- Duplicates are linked to the original and stay visible and open.

> **Verification status.** The Groq transport was tested with an injected `fetch`; the live Groq call has not been verified against api.groq.com (§12.4, §13). Without a valid `GROQ_API_KEY` the application runs in keyword fallback mode.

11. Security, Privacy and UX Decisions

11.1 Security and privacy controls

Area	Control
Secrets	`GROQ_API_KEY`, `DB_PASSWORD`, `JWT_SECRET` only in backend environment; `.env` git-ignored; `.env.example` holds names only; a test scans the repository and browser bundle for secrets.

Area	Control
AuthN / AuthZ	JWT (HS256, 8 h); every route declares allowed roles; residents are confined to their own flat; matrix test covers each route (403 / 401).
Input	Length, enum, date and phone validation; malformed JSON → 400; 6 MB body cap; photo type and size checks; parameterised SQL only.
Rate limiting	Per-IP API and OTP limits, per-user complaint creation, public form limit; 429 with Retry-After; local budget for Groq calls.
Logging	Keys, passwords and complaint text are never logged; database description prints no password.
Privacy (DPDP 2023)	Privacy notice on gate screens; minimal data to the AI provider; visitor photos deleted after 90 days by an hourly job (audited); audit trail of staff actions.
Headers	`X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy`, `Cache-Control: no-store` on the API.

11.2 UI / UX decisions

- **Preserved:** branding, cinematic landing page, navigation, dashboards, responsive layout, three languages.
- **Added:** in the help-desk complaint drawer — classification method badge (AI / keyword fallback / corrected by staff), confidence and explanation, manual-review banner with a one-tap Confirm, priority control, correction audit trail. A safety banner states the escalation was by rule. Residents never see AI internals (reasons, confidence, review flags).
- **States:** loading skeletons, empty states, success toasts and inline errors on every data screen; the mobile top bar was fixed for narrow phones. Verified at 390, 820 and 1440 px with no horizontal overflow on the help desk.
- **Principle:** most important information within two taps; every automated decision is visible and reversible by a person.
- **Demo transparency:** a “DEMO MODE: no SMS is sent and no real payment is taken” banner is shown on every screen in the captures.

12. Testing and Quality Assurance

Automated tests are run with `npm test` (Node’s built-in runner). The database tests execute `schema.sql` on SQLite (`node:sqlite`), so the automated suite is not a MySQL test run. MySQL was verified separately by the local steps in §12.3, and neither result covers the Aiven/Vercel production deployment (§12.5).

12.1 Automated test results

Total tests	Passed	Failed	Skipped
99	**98**	**0**	**1**

The skipped test is the live Groq API test, which requires a valid `GROQ_API_KEY`. It was not run, so live Groq behaviour is not covered by this result.

The suite runs on the SQLite test environment. The 98 passed tests therefore do not show that every test ran against MySQL. An earlier revision recorded that the repository-hygiene test and the `.env` loader were fixed for Windows paths; that fix was verified on Linux at the time.

12.2 Test coverage

Required area	Coverage (test files)
Database connectivity	round-trip query, engine reported, migration recorded, configuration from environment, failure without `DATABASE_URL` (`database.test.js`)
Resident and billing persistence	CRUD across independent repository instances; brief sample data reproduced from rows; bill generation idempotent (`database.test.js`, `billing.test.js`)
Payment matching	outcomes on the sample file; UPI+amount → review; overpayment → review; import never posts (`matching.test.js`)
Duplicate UTR / payment reference	blocked in code and by unique keys; same bank transaction cannot post twice; concurrent postings → exactly one commit; atomic rollback (`matching.test.js`, `billing.test.js`, `database.test.js`)
AI classification	valid answer stored with method and model; request hygiene (no PII, key only in header) (`ai.test.js`)
Invalid AI response	validator rejects each malformed field; retry with feedback; two failures → fallback; unknown category never stored (`ai.test.js`)
API failure fallback	network, timeout, 401, 429, 500, empty body; no key; call budget (`ai.test.js`)
Emergency escalation	fire/gas/lift/shock stay Emergency even if the model says Low, fails or returns junk; prompt injection; cannot be downgraded; Hindi (`ai.test.js`, `triage.test.js`)
Low-confidence review	flagged with reason; queue; confirm or correct with audit (`ai.test.js`)
Role-based authorisation	route × role matrix, cross-flat isolation, JWT tamper/expiry (`security.test.js`, `api.test.js`)

Browser checks (Playwright/Chromium against the SQLite dev runner): resident sign-in, raising a gas-leak complaint, help-desk escalation banner, classification card, refusal to lower an emergency, and no horizontal overflow at mobile, tablet and desktop widths.

12.3 Local MySQL verification

On 5 November 2026 the application was verified against a local MySQL server. The results are listed below. No credentials or connection strings are recorded in this document.

Check	Result
MySQL Server version	8.0.46
Database	`nestora_db`
`npm run db:init`	Completed successfully.
Schema verification	Confirmed 26 tables.
`users` table	9 records.
`complaints` table	11 records.
`npm start`	Connected to MySQL successfully.
`GET /api/health`	Returned the JSON response shown below the table.
`npm run db:seed`	Did not insert new demo data, because the database already contained data. Seeding is not reported as successful.

{"ok":true,"database":"mysql","time":"2026-11-05T08:31:23.744Z"}

These checks show that the MySQL adapter connected to the local server, that the schema initialised with the expected 26 tables, and that the application reported a healthy MySQL connection. They did not include a run of the automated test suite against MySQL. Because `db:seed` inserted nothing, seeding on an empty MySQL database has not been verified. No destructive database reset was run for this verification, and none is recommended.

12.4 Groq AI status

Groq integration is implemented as an optional, configurable feature (§10). **Live Groq API integration has not yet been verified.** The live Groq test is skipped because it needs a valid `GROQ_API_KEY`, and no live request to api.groq.com has been recorded. The Groq transport has been tested only with an injected `fetch`. Without a valid API key the application uses keyword-based fallback mode, makes no network calls to Groq, and stores complaints with the method `keyword_fallback`. This document makes no claim that live AI functionality has been tested.

12.5 Aiven and Vercel production status

The live application on the cover page is deployed on Vercel. The local MySQL verification in §12.3 does not prove that the Aiven/Vercel production database connection is working: it exercised a local server only, not the production database settings or data. This revision records no verification of the production database and no verification of production AI integration. Both must be checked separately before they are described as working.

12.6 Executed and not executed

Item	Status
Automated test suite (SQLite test environment)	Executed: 99 tests, 98 passed, 0 failed, 1 skipped.
Browser checks (SQLite dev runner)	Executed, as recorded above.
Local MySQL: `db:init`, schema check, `npm start`, `/api/health`	Executed and passed (§12.3).
Local MySQL: `db:seed`	Executed; no new demo data inserted because data already existed. Not reported as verified.
Automated tests against MySQL	Not executed.
Live Groq API call	Not executed; the live-Groq test was skipped.
Aiven/Vercel production database	Not verified.
Production AI integration	Not verified.

13. Assumptions, Limitations and Future Improvements

13.1 Assumptions

- MySQL 8.0.16 or later (CHECK constraints are enforced) with InnoDB and utf8mb4. The local verification in §12.3 used MySQL 8.0.46.
- One township, one currency (INR); late-fee policy is whatever the committee configures — none by default.
- The mock OTP and test-mode payments are intentional for evaluation; no real money or card data is handled.
- Hosted-model catalogues change: the default model is configurable and should be confirmed against Groq's current list.
- The delivered stack (Node, vanilla JS) is retained from the earlier build.

13.2 Current limitations (implemented behaviour and unverified items)

Limitation	Impact
Unverified: live Groq API, Aiven/Vercel production database and production AI integration	The Groq transport was tested only with an injected `fetch`; without a valid key the keyword fallback is used. Local MySQL verification (§12.3) does not cover the production deployment.
Automated tests run on SQLite, not MySQL	SQLite runs the identical `schema.sql`, but MySQL-specific behaviour is covered only by the manual local checks in §12.3.
`db:seed` not verified on an empty MySQL database	The seed command inserted nothing on the local database because data already existed.

Limitation	Impact
Demo OTP and test-mode payments	No SMS is sent and no real payment is taken.
In-memory OTP and rate-limit counters	Not shared across multiple instances.
Photos stored in the database	Acceptable at demonstration scale only.
Analytics read rows into the application	Not tuned for full 2,400-flat scale.
Keyword fallback is English/Hindi/Marathi keywords only	Accuracy of the fallback has not been measured.
No schema migration framework	Only the idempotent `schema.sql` is used.
Screenshots	Captured with demonstration data; they are not evidence of MySQL or production verification. The mobile capture is a single device-emulation view (§14).

13.3 Future improvements (not implemented)

Improvement	Purpose
Run `npm test` with `GROQ_API_KEY` set	Verify the live Groq path.
Verify the Aiven/Vercel production deployment (database connection and AI integration)	Confirm production behaviour; local results do not show it.
Run the database tests against a dedicated MySQL test instance	Cover MySQL-specific behaviour in automated tests.
SMS gateway and PCI-compliant payment gateway	Replace the demo OTP and test-mode payments.
Redis for OTP and rate-limit state	Required before running more than one instance.
Object storage with signed URLs	Replace photos stored in the database.
SQL aggregation / materialised views	Analytics at full scale.
Evaluate stored help-desk corrections	Tune confidence thresholds and the keyword fallback.
Versioned migrations (Flyway-style)	Manage later schema changes.

14. Mobile Responsive Design

Figure 9 shows the resident dashboard in a browser device-emulation view configured as Samsung Galaxy A55 at 360 × 800 (the emulation toolbar is visible at the top of the capture). It is an emulated view, not a photograph of a physical device. The observations below are limited to what is visible in the capture.

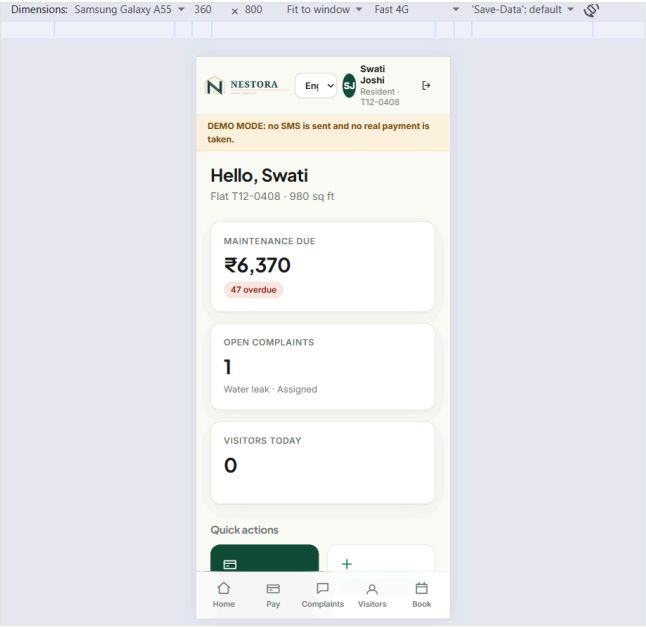


Figure 9: NESTORA Resident Dashboard – Samsung Galaxy A55 Mobile Responsive View (360 × 800).

Element	Visible behaviour at 360 px
Top navigation	The desktop navigation links are not shown; the top bar holds the logo, a compact language selector (label truncated to “Eng”), the user avatar with name, role and flat, and a sign-out icon. The name and flat wrap onto several lines.
Demo banner	The demonstration-mode notice wraps onto two lines at the full width.
Dashboard cards	Cards are stacked in a single column at full width: **Maintenance due** (₹6,370 with an overdue badge), **Open complaints** (1, “Water leak · Assigned”) and **Visitors today** (0). Flat and area (T12-0408 · 980 sq ft) are shown in the greeting.
Quick actions	A “Quick actions” heading with a two-column tile grid, partly below the visible fold in this capture.
Bottom navigation	A fixed bar with five icon-and-label items — Home, Pay, Complaints, Visitors, Book — replaces the top navigation links shown on desktop (§6, Figure 4).

> **Scope of verification.** This section documents one captured viewport (360 × 800). Earlier checks recorded in §11 and §12 cover 390, 820 and 1440 px for the help desk and mobile, tablet and desktop widths in browser checks. No claim is made that every device or screen size has been tested.